Generative Latent Models for DeepSDF: Gaussian vs. Diffusion in the Small-Data Regime

Amit Benbenishti
Deep Learning for 3D Computer Vision
Hebrew University of Jerusalem

Abstract

DeepSDF (Park et al., 2019) represents a shape category with one MLP and a per-shape latent code, but it does not learn a *distribution* over those codes, so drawing new codes at random rarely decodes to a valid shape. Latent diffusion (3D-LDM (Nam et al., 2022)) fixes this at full ShapeNet scale, but its benefit over a trivial Gaussian prior in the low-data, low-capacity regime is unstudied. I train a DeepSDF auto-decoder on ShapeNet chairs, freeze it, and fit three priors over the resulting codes-a multivariate Gaussian, a Gaussian mixture (GMM), and a DDPM denoising the codes directly-as the dataset shrinks, $N \in \{10, 50, 150\}$. Diffusion does not pay for itself here: the GMM attains the best Coverage at every N and the Gaussian is close behind, while the DDPM wins no cell, is worst on most generation metrics at $N = 150$, and at $N = 10$ yields an extractable surface for only 15% of its samples against 100% for the Gaussian. I also report a pitfall that is easy to misdiagnose in a small-data study like this one: raw ShapeNet meshes are not watertight (closed) solids, which corrupts the inside/outside sign DeepSDF trains on while the training loss still looks healthy.

1 Introduction

A signed distance function (SDF) represents a surface implicitly, as the zero level set of a function $f : \mathbb{R}^3 \to \mathbb{R}$ that outputs the distance from a point to the surface. DeepSDF (Park et al., 2019) learns one such function per shape *category*: a single MLP $f_\theta(\mathbf{z}, \mathbf{x})$ that takes a per-shape code $\mathbf{z}$ alongside the query point $\mathbf{x}$, so different codes trace out different shapes from the same network. Rather than using an encoder to produce $\mathbf{z}$, DeepSDF treats each code as a free parameter and optimizes it jointly with the network weights - an *auto-decoder*. The signed distance is one of several implicit targets a network can be trained to predict: Occupancy Networks (Mescheder et al., 2019) and IM-Net (Chen and Zhang, 2019) predict inside/outside occupancy instead. This paper keeps that choice of representation and decoder fixed and asks a question one level up, about the codes rather than the network that decodes them.

DeepSDF's L_2 code regularizer is the MAP estimate under a zero-mean Gaussian prior, so one could in principle sample $\mathbf{z} \sim \mathcal{N}(0, I)$ and decode; in practice the learned codes do not fill an isotropic Gaussian and naive sampling breaks. Generative shape modeling has largely tackled this by working directly on point clouds or voxels, with GANs and VAEs (Achlioptas et al., 2018) and, more recently, diffusion models (Poole et al., 2023). 3D-LDM (Nam et al., 2022) instead brings that diffusion machinery inside the DeepSDF latent space: it trains a diffusion model over the per-shape codes directly, at full ShapeNet scale, following DDPM (Ho et al., 2020).

Both operate at scale. My question is narrower and controlled: *on a real ShapeNet chair subset with small N, when does a DDPM over DeepSDF codes beat simpler priors?* I hold Stage 1 fixed, vary only the Stage 2 generator, and report reconstruction and distribution-level generation metrics. This comparison is the paper's main contribution, and the answer turns out to be negative for diffusion: at small N, a plain Gaussian or a Gaussian mixture matches or beats the DDPM. That is a useful result rather than a disappointing one, because it suggests 3D-LDM's advantage over simpler priors comes from the amount of data it was trained on, not from diffusion being intrinsically better at modeling the latent space. All training code is my own.

2 Method

2.1 Stage 1: DeepSDF auto-decoder

Each of the N shapes gets a code $\mathbf{z}_i \in \mathbb{R}^D$ initialized from $\mathcal{N}(0, 0.01^2)$, sharing a decoder f_θ of 8 hidden layers, width 256, ReLU, with $[\mathbf{z}; \mathbf{x}]$ reinjected at layer 4, weight normalization, and an unsquashed linear output. I use *geometric initial-*

ization (Atzmon and Lipman, 2020; Gropp et al., 2020) so the untrained field is an approximate sphere SDF; without it the decoder can settle into a constant field that never crosses zero. Per mesh (normalized into the unit sphere) I sample 8000 points, 90% near the surface (perturbed by $\sigma \in \{0.005, 0.02\}$) and 10% uniform in $[-1, 1]^3$. Following DeepSDF, I minimize

$$\mathcal{L} = \frac{1}{|B|} \sum_{(i,\mathbf{x})\in B} \left| \hat{f}_\theta(\mathbf{z}_i, \mathbf{x}) - \hat{s}(\mathbf{x}) \right| + \lambda \frac{1}{|B|} \sum_i \|\mathbf{z}_i\|_2^2, \tag{1}$$

where $\hat{a} \equiv \mathrm{clamp}(a, \delta)$ with $\delta = 0.1$ and $\lambda = 10^{-4}$. Clamping concentrates capacity on the near-surface band that Marching Cubes actually reads. Weights and codes are optimized jointly with Adam. To realize a sampled code I evaluate f_θ on a grid over $[-1, 1]^3$ and run Marching Cubes at level 0.

2.2 Making the SDF sign meaningful

The label in Eq. (1) is a *signed* distance, so it presupposes a closed surface. Raw ShapeNetCore OBJs are not: they are assembled *shells*, and a representative chair from synset `03001627` decomposes into 313 disconnected components never welded into one manifold. On such input `trimesh`'s winding-number query has no coherent interior to test against, so only $\approx 1\%$ of sampled points were labelled inside, versus $\approx 38\%$ at the same locations after repair.

With nearly every near-surface label positive, the network fits an unsigned field with no interior, so Marching Cubes returns fragments - while the point-wise loss stays low, since the decoder faithfully fits whatever labels it was given. No downstream tuning can repair a corrupted label, so I instead voxelize each non-watertight mesh at pitch $1/64$, fill the interior, and re-mesh with Marching Cubes, yielding a single closed solid regardless of input topology, at the cost of detail thinner than the pitch.

2.3 Stage 2: priors over frozen codes

I freeze f_θ and $\{\mathbf{z}_i\}$ and fit three generators spanning a minimal-to-expressive spectrum. The **Gaussian** is a full-covariance $\mathcal{N}(\boldsymbol{\mu}, \Sigma)$ fit by maximum likelihood with a small ridge. The **GMM** is a full-covariance mixture fit by EM (implemented from scratch), with K capped so each component receives at least 5 codes on average, giving $K = 2$ at $N = 10$ and $K = 5$ otherwise. The **DDPM** denoises the D-dimensional codes directly with the noise-prediction objective (Ho et al., 2020) under a cosine schedule (Nichol and Dhariwal, 2021), using a small MLP $\boldsymbol{\epsilon}_\phi(\mathbf{z}_t, t)$ with a sinusoidal time embedding, and generates by ancestral sampling from $\mathcal{N}(0, I)$.

3 Experiments

Setup. All cells use ShapeNetCore chairs (Chang et al., 2015) (`03001627`). The proposal specified a full $N \times D$ grid; I instead sweep $N \in \{10, 50, 150\}$ at a fixed $D = 16$ and treat the latent dimension as a single ablation ($D = 32$ at $N = 50$), which keeps the generator comparison on one clean axis. Each cell trains Stage 1 for 8000 Adam iterations (learning rate 10^{-3}, 48 shapes and 1024 points per shape per step), then fits the three priors on the frozen codes; the DDPM uses 200 timesteps and trains for 3000 iterations. Reconstruction is measured on 10 training shapes at Marching-Cubes resolution 48^3 (IoU at 32^3). For generation I draw 40 samples per prior against 40 held-out chairs and report Coverage, MMD, and 1-NN accuracy (Achlioptas et al., 2018), all computed with Chamfer distance between surface point clouds, plus the *valid ratio*: the fraction of sampled codes that yield an extractable surface at all. Valid ratio is a first-class result here, not a diagnostic, because Coverage and MMD are computed only over samples that produced a mesh, so a prior can look competitive while silently failing on most of its draws. `scripts/run_grid.py` reproduces every cell on one Colab GPU in about 2.5 hours.

The simplest priors win. Table 1 and Fig. 1 report the sweep. The GMM attains the highest Coverage at every N (0.325, 0.300, 0.400) and the Gaussian is close behind (0.250, 0.275, 0.325); the ordering GMM $\geq$ Gaussian $>$ DDPM holds in two of three cells and the three tie within 0.025 at $N = 50$. The DDPM wins no cell. At $N = 10$ it fails outright, decoding to a surface for only 15% of its samples, so its Coverage and MMD there are computed over six samples out of forty and are not comparable to the other rows. At $N = 150$, under the fixed training budget used for the whole sweep, it is worst on three of the four generation metrics (Coverage, MMD, 1-

Table 1: Reconstruction and generation metrics. Recon-CD and MMD are lower-is-better; IoU and Coverage higher-is-better; 1-NN accuracy is best near 0.5. Reconstruction is a Stage-1 property and is therefore shared by the three generators within a cell.

N	D	Gen.	Recon-CD	IoU	Coverage	MMD	1-NN	Valid
Main sweep ($D = 16$)								
10	16	ddpm	0.0151	0.784	0.100	0.0331	0.978	0.15
10	16	gaussian	0.0151	0.784	0.250	0.0257	0.938	1.00
10	16	gmm	0.0151	0.784	0.325	0.0244	0.949	0.97
50	16	ddpm	0.0361	0.736	0.275	0.0229	0.938	1.00
50	16	gaussian	0.0361	0.736	0.275	0.0300	0.900	1.00
50	16	gmm	0.0361	0.736	0.300	0.0294	0.975	1.00
150	16	ddpm	0.0122	0.683	0.225	0.0424	0.988	1.00
150	16	gaussian	0.0122	0.683	0.325	0.0310	0.975	1.00
150	16	gmm	0.0122	0.683	0.400	0.0296	0.963	1.00
Latent-dimension ablation								
50	32	ddpm	0.0145	0.814	0.250	0.0506	0.986	0.85
50	32	gaussian	0.0145	0.814	0.225	0.0320	0.925	1.00
50	32	gmm	0.0145	0.814	0.225	0.0279	0.963	1.00

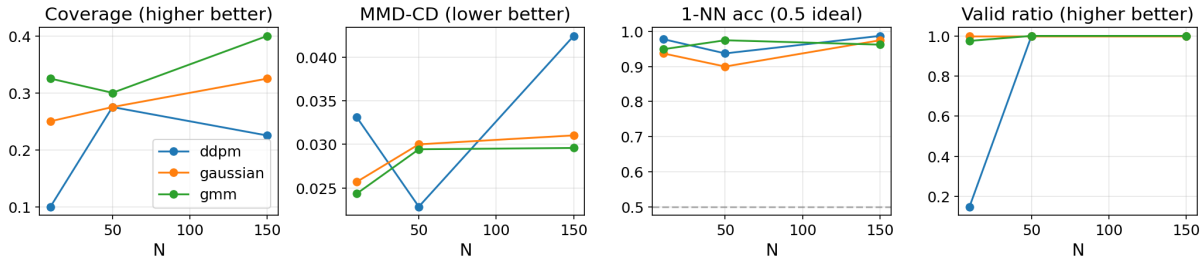

Figure 1: Generation metrics vs. number of training shapes N, per prior, at $D = 16$. The dashed line marks the ideal 1-NN accuracy of 0.5.

NN); a matched-budget control below (*Confirmatory control*) closes the Coverage gap there without moving MMD or 1-NN. Its one strength, the best MMD in the study (0.0229 at $N = 50$), sits beside the two worst (0.0424 at $N = 150$; 0.0506 in the ablation), which reads as variance rather than capability. The Gaussian–GMM gap is real but modest (0.075, 0.025, 0.075 in Coverage, always in the same direction): a mixture earns its two extra lines, whereas the diffusion model costs an entire second training loop and is actively harmful at the extremes of the sweep.

Everything is far from real. 1-NN accuracy spans 0.900–0.988 and never approaches the ideal 0.5; the best value is the Gaussian at $N = 50$. Generated and real chairs remain trivially separable in every configuration, so the ranking above describes degrees of failure, not a contest between models that work. This bounds how much weight the Coverage and MMD comparison can carry.

Trend with N. Under the fixed 8000-iteration budget used for every cell, more shapes cut two ways: the prior sees a larger sample, but Stage 1 has to spend that same budget across more geometry. Reconstruction IoU falls monotonically ($0.784 \rightarrow 0.736 \rightarrow 0.683$), yet Coverage rises over the same range for the Gaussian and the GMM. The DDPM does not share this: it improves sharply from $N = 10$ to $N = 50$ ($0.100 \rightarrow 0.275$ Coverage, $0.15 \rightarrow 1.00$ valid) and then degrades at $N = 150$, so its problem is not sample size alone. Reconstruction Chamfer is not monotone (0.0151, 0.0361, 0.0122) and I read the IoU trend as the reliable one: Chamfer is an unbounded average over ten shapes, so one poor decode can dominate it, whereas IoU is bounded in $[0, 1]$. The obvious reading of the IoU trend - a fixed-width decoder running out of room for 150 chairs - turns

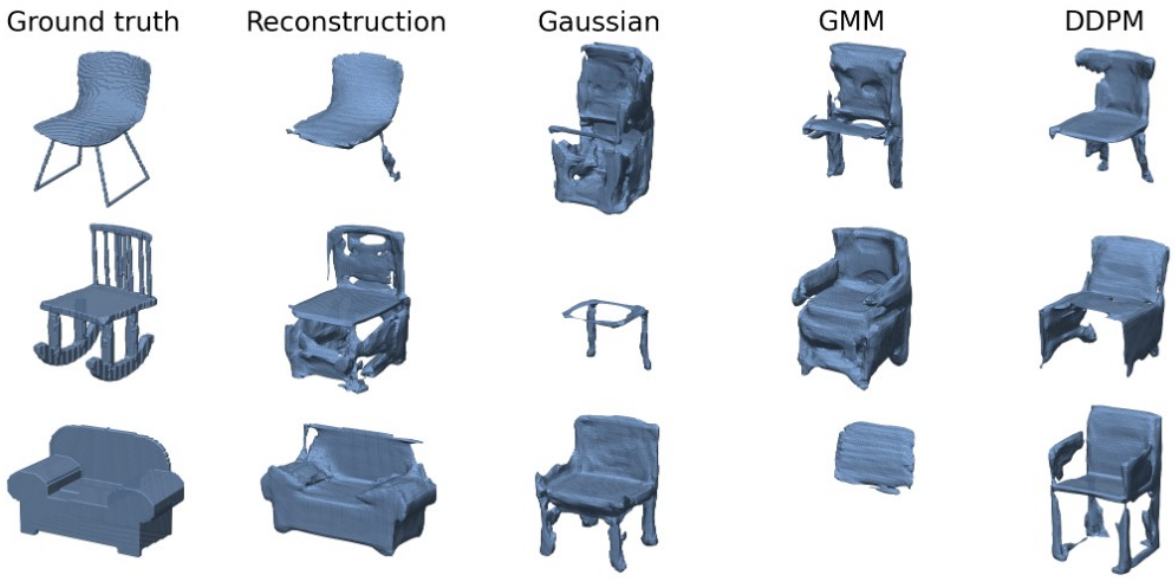

Figure 2: Qualitative results at $N = 150$, $D = 16$, as shaded solids. Columns: ground-truth chair, its reconstruction from the learned code, then one sample each from the Gaussian, GMM, and DDPM priors; rows are independent draws. Reconstructions preserve the coarse massing but lose thin structure - the sled legs of row 1 and the back slats of row 2 - and fare best on the bulky armchair of row 3. All three priors emit chair-like solids at this N, consistent with the valid ratio of 1.00 in Table 1, with visible holes and melted detail.

out to be wrong, as the next paragraph shows.

Confirmatory control: capacity or training budget? `random_batch` in Stage 1 draws `shapes_per_batch`$= 48$ shapes *without replacement* every step, and that count is fixed across the whole sweep, so a given shape's code is updated on only a $\min(48, N)/N$ fraction of steps: 100% at $N = 10$, 96% at $N = 50$, but 32% at $N = 150$ - roughly $3\times$ fewer gradient updates per shape than the other two cells, confounded with the capacity story above by construction. Re-training $N = 150, D = 16$ alone with $3\times$ the iterations (24000, matching $N = 50$'s per-shape budget) reverses the IoU trend: Chamfer falls to 0.0108 and IoU rises to 0.789 - the best reconstruction in the entire study, ahead of $N = 10$ and $N = 50$ at their original budget (Fig. 3). The decoder does not run out of capacity for 150 chairs; most of the earlier degradation was the training-budget confound, not the fixed decoder. Generation does not resolve as cleanly: Coverage converges to 0.300 for all three priors under the matched budget (Gaussian $0.325 \to 0.300$, GMM $0.400 \to 0.300$, DDPM $0.225 \to 0.300$), so the better-trained codes are, if anything, *harder* for the classical priors to cover - plausibly because their norm grows further ($\|\mathbf{z}\| = 0.517$, above the 0.29–0.41 range of the main sweep) as the codes differentiate more per shape. MMD and 1-NN still favor Gaussian and GMM over the DDPM at the matched budget (0.0349/0.0380 vs. 0.0404; 0.938/0.963 vs. 1.000), so the paper's main conclusion is unaffected, but the specific claim that DDPM is worst on *every* generation metric at $N = 150$ does not survive a fair training budget.

Latent dimension, and a correction. Doubling D to 32 at $N = 50$ helps Stage 1 - Chamfer $0.0361 \to 0.0145$, IoU $0.736 \to 0.814$, the best reconstruction in the study - and hurts Stage 2, with Coverage down for all three priors and the DDPM degrading sharply (MMD $0.0229 \to 0.0506$, valid $1.00 \to 0.85$). Wider codes give Stage 1 more room per shape, but Stage 2 must then estimate a density in $\mathbb{R}^{32}$ from the same 50 points. This *contradicts* the pre-repair measurement that motivated fixing $D = 16$, where the same change moved Chamfer from 0.129 to 0.540. I report the reversal rather than drop it, because it supports the diagnosis of Section 2.2: before repair, capacity was rewarded for fitting a sign field that was largely arbitrary, so more capacity meant a better fit to worse labels. Once the labels describe a closed solid, capacity behaves as expected. The caveat is that I re-tested only the latent dimension; the width-512 decoder that showed the same inversion was never

Figure 3: Same $N = 150, D = 16$ shapes reconstructed at the main sweep's 8000-iteration budget (row 2) and the iteration-matched 24000-iteration control (row 3), against ground truth (row 1). Thin structures are somewhat better preserved under the matched budget, consistent with the $0.683 \rightarrow 0.789$ IoU improvement reported below, though both budgets still lose fine detail relative to ground truth.

re-run post-repair.

What the samples look like. Figure 2 is the check the metrics cannot provide, since Chamfer distance is forgiving of a shape with roughly the right mass in roughly the right places. Reconstruction degrades exactly where the repair of Section 2.2 predicts: the bulky armchair survives well, while the sled legs and back slats - structures thinner than the $1/64$ voxel pitch - are thickened, holed, or lost. That is what holds 1-NN accuracy near 1.0 regardless of the prior, and it is a Stage-1 ceiling no generator can lift.

The priors are harder to separate by eye than the table suggests. All three emit chair-like solids with recognizable seats and backs, all three show the same melted surfaces and holes, and all three occasionally emit something degenerate: a spindly frame with no seat (Gaussian, row 2), a flattened slab (GMM, row 3). This seems like the honest reading rather than a limitation of the figure. The measured gaps at $N = 150$ are *distributional* - Coverage 0.400 for the GMM against 0.225 for the DDPM - and three draws per prior cannot resolve a difference in how much of the reference set gets matched. Per-sample plausibility, which is what the eye judges here, is roughly comparable. The contrast with $N = 10$ is the point: there the DDPM's deficit was catastrophic and obvious, producing no surface at all, whereas at $N = 150$ it has become a diversity deficit that only the metrics expose.

Training pathologies. Two failures cost most of the project's time and are worth recording. Geometric initialization zeroes the latent columns of the first layer, so $\partial \mathcal{L}/\partial \mathbf{z}$ is exactly zero at step 1 and the only gradient reaching the codes is the regularizer pulling them to the origin; several runs collapsed to $\|\mathbf{z}\| \approx 0.0005$, after which the decoder

ignores $\mathbf{z}$ and emits one average shape. Warming up λ from zero fixes it. Separately, a width-512 decoder did not tolerate the learning rate that worked at width 256 and needed 3×10^{-4}. I also tried the IGR eikonal penalty (Gropp et al., 2020): at its default weight of 0.1 it destroyed training, because the clamped data term is $O(\delta) = O(10^{-2})$ while the unclamped eikonal term is $O(1)$, so the regularizer dominates by an order of magnitude. In the final runs Stage 1 was healthy in all four cells, with final loss 0.0024–0.0049 and code norms growing monotonically to 0.29–0.41.

4 Conclusion

I compared Gaussian, GMM, and DDPM priors over frozen DeepSDF codes on ShapeNet chairs as the dataset shrank. Diffusion is not worth its complexity at this scale: the GMM led Coverage at every N, the Gaussian trailed closely, and the DDPM won no cell, was worst on most generation metrics at $N = 150$ (a training-budget-matched control closes the Coverage gap there but leaves MMD and 1-NN unchanged), and decoded to a surface only 15% of the time at $N = 10$. Since 1-NN accuracy never fell below 0.900 against an ideal of 0.5, what is being ranked is degrees of failure - but the ranking is consistent, and it locates the benefit of latent diffusion in the data scale it was demonstrated at rather than in the mechanism.

The more transferable lesson is that the failure which dominated my results presented as an optimization problem while being a data problem (Section 2.2), and that its most confusing symptom - capacity simultaneously lowering training loss and worsening geometry - vanished once the labels were repaired. The monitored quantity moved reassuringly throughout. On a pipeline that ends in Marching Cubes, point-wise loss is a weak proxy for the surface, and geometry has to be inspected directly. A second instance of the same lesson showed up inside the N-sweep itself: what read as a capacity limit (*Trend with* N) was, once tested directly, mostly a training budget confounded with N by an unchanged batch size. Neither failure was visible in the number the optimizer was minimizing; both needed a targeted control that held something else constant instead.

Limitations include the single synset, the cost of mesh repair, and the modest sample counts behind the generation metrics (40 against 40), which limits how finely Coverage and 1-NN accuracy resolve. Future work could scale N within chairs, retune the optimizer for wider decoders rather than fixing width, and add a flow-based prior as a fourth point on the spectrum.

References

Panos Achlioptas, Olga Diamanti, Ioannis Mitliagkas, and Leonidas Guibas. 2018. Learning representations and generative models for 3d point clouds. In *International Conference on Machine Learning (ICML)*.

Matan Atzmon and Yaron Lipman. 2020. SAL: Sign agnostic learning of shapes from raw data. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

Angel X. Chang, Thomas Funkhouser, Leonidas Guibas, Pat Hanrahan, Qixing Huang, Zimo Li, Silvio Savarese, Manolis Savva, Shuran Song, Hao Su, Jianxiong Xiao, Li Yi, and Fisher Yu. 2015. ShapeNet: An information-rich 3d model repository. *arXiv preprint arXiv:1512.03012*.

Zhiqin Chen and Hao Zhang. 2019. Learning implicit fields for generative shape modeling. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

Amos Gropp, Lior Yariv, Niv Haim, Matan Atzmon, and Yaron Lipman. 2020. Implicit geometric regularization for learning shapes. In *International Conference on Machine Learning (ICML)*.

Jonathan Ho, Ajay Jain, and Pieter Abbeel. 2020. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Lars Mescheder, Michael Oechsle, Michael Niemeyer, Sebastian Nowozin, and Andreas Geiger. 2019. Occupancy networks: Learning 3d reconstruction in function space. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

Gimin Nam, Mariem Khlifi, Andrew Rodriguez, Alberto Tono, Linqi Zhou, and Paul Guerrero. 2022. 3D-LDM: Neural implicit 3d shape generation with latent diffusion models. *arXiv preprint arXiv:2212.00842*.

Alexander Quinn Nichol and Prafulla Dhariwal. 2021. Improved denoising diffusion probabilistic models. In *International Conference on Machine Learning (ICML)*.

Jeong Joon Park, Peter Florence, Julian Straub, Richard Newcombe, and Steven Lovegrove. 2019. DeepSDF: Learning continuous signed distance functions for shape representation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

Ben Poole, Ajay Jain, Jonathan T. Barron, and Ben Mildenhall. 2023. DreamFusion: Text-to-3d using 2d diffusion. In *International Conference on Learning Representations (ICLR)*.

A Original Proposal

The approved project proposal is reproduced below.

Title. Generative Latent Models for DeepSDF: Gaussian vs Diffusion in the Small-Data Regime. *Author:* Amit Benbenishti.

Problem definition. DeepSDF learns one latent code per shape, but does not learn a distribution over codes—random codes decode to invalid shapes. I train a generative model over the latent space (not over shapes directly) and study which simple prior works when data and capacity are limited.

Approach. *Stage 1:* train a DeepSDF auto-decoder on a shape collection; collect latent codes $\{z_i\}$. *Stage 2 (frozen decoder):* fit two generators on $\{z_i\}$—(1) Gaussian (mean + covariance), (2) DDPM diffusion over z (same noise-prediction setup as HW3, applied to vectors). *Inference:* sample z from a generator $\rightarrow$ decode with $f(z, x) \rightarrow$ SDF $\rightarrow$ mesh. Sweep $N \in \{10, 50, 150\} \times D \in \{16, 32\}$ and compare generators. Goal is not SOTA—it is to identify when diffusion is worth the extra complexity over a Gaussian. All training code is my own.

Motivation. 3D-LDM uses latent diffusion over DeepSDF codes at full ShapeNet scale; naive Gaussian sampling from the same latent space often fails. I study the minimal-resource regime (small N, small D, Colab) where that tradeoff is unknown.

Validation. *Data:* ShapeNet subset ($\sim$50–200 meshes) or synthetic shapes + `stanford_bunny.obj`. *Metrics:* Chamfer + IoU (reconstruction); Coverage, MMD, 1-NN accuracy (generation); qualitative samples and interpolations. *Mid-point checks:* (1) overfit one shape, (2) train DeepSDF on $\sim$10 shapes and extract codes, (3) sample one mesh from Gaussian and one from diffusion. *Final:* full $N \times D$ grid + generator comparison + degradation curves, including negative results (e.g., Gaussian matches diffusion at small N).

Related work. DeepSDF (Park et al., 2019)—auto-decoder and latent codes (Stage 1). 3D-LDM (Nam et al., 2022)—latent diffusion over DeepSDF codes at scale; does not study small N/D or Gaussian vs diffusion explicitly. DreamFusion / DDPM (Poole et al., 2023; Ho et al., 2020)— diffusion training (HW3 connection).